Guía 1 de 3 — Funciones, decisiones y errores

DPL1046 · Clase 7 · Para repasar en casa, a tu ritmo.

Hasta ahora escribiste scripts que funcionan en tu computador. Para convertirlos en un **programa** ordenado y confiable necesitas tres ideas: **funciones**, **decisiones y repeticiones** (`if` , `for`) y **manejo de errores** (`try / except`).

> Antes de empezar, haz tu rutina de inicio (Guía 1 de la Clase 2) con el `.venv` activado.

---

1. Funciones: una receta con nombre

Una **receta de cocina** tiene un nombre ("tallarines con salsa"), unos ingredientes que le pasas y entrega un plato. La escribes una vez y la usas todas las veces que quieras con otros ingredientes. Una **función** es eso.

```
def saludar(nombre):
    return "Hola " + nombre

print(saludar("Ana"))      # Hola Ana
print(saludar("Luis"))     # Hola Luis
```

Parte	Qué es
`def`	"voy a definir una receta"
`saludar`	el **nombre** de la función
`(nombre)`	el **parámetro**: el ingrediente que recibe
`:`	los dos puntos: "ahora viene lo que hace"
la **sangría**	lo corrido a la derecha pertenece a la función
`return`	lo que **devuelve**

> Definir una función no ejecuta nada. **Hay que llamarla**: su nombre y, entre paréntesis, lo que necesita.

Con datos reales

```
def calcular_dias(compra, entrega):
    """Devuelve los días que pasan entre dos fechas (escritas como texto)."""
    dias = (pd.to_datetime(entrega) - pd.to_datetime(compra)).days
    return dias

def cargar_tabla(nombre, ruta=RUTA):
    """Lee una tabla CSV de la carpeta de datos y la devuelve."""
    return pd.read_csv(ruta + nombre)

print(calcular_dias("2017-10-02 10:56:33", "2017-10-10 21:25:13"))   # 8
print(calcular_dias("2018-07-24", "2018-08-07"))                     # 14
orders = cargar_tabla("olist_orders_dataset.csv")
```

- `"""..."""` (la primera línea dentro) es el **docstring**: una nota que explica qué hace la función.
- `ruta=RUTA` es un **valor por defecto**: si no lo indicas, usa `RUTA`. Es como el "sin azúcar" del café.
- La **misma** función se usa con otros datos: escribes el cálculo una vez.
- Sin `return`, la función hace su trabajo pero **no devuelve nada**.

---

2. Decisiones y repeticiones

- **`if` es el semáforo:** si está verde avanzo; si no, espero.
- **`for` es pasar lista:** hago lo mismo con cada alumno.
- **`try` es el paraguas:** lo llevo por si llueve.

Las tres siguen el mismo patrón: una línea que **termina en dos puntos** y, debajo, lo que va **con sangría**.

```
def clasificar_entrega(dias):
    """Clasifica una entrega según los días que tardó."""
    if dias <= 7:
        return "rápida"
    elif dias <= 20:
        return "normal"
    else:
        return "lenta"

for dias in [3, 12, 45]:
    print(dias, "días →", clasificar_entrega(dias))
```

Resultado: `3 días → rápida`, `12 días → normal`, `45 días → lenta`.

- `if dias <= 7:` → "si los días son siete o menos". `<=` es "menor o igual". Al hacer `return`, la función **termina ahí**.

- `elif` → "si no, pero si…". Solo se mira si lo anterior fue falso.
- `else` → "en cualquier otro caso". No lleva condición.
- `for dias in [3, 12, 45]:` → "para cada número de esta lista"; `dias` vale 3, luego 12, luego 45.

Sigue el 12 a mano: no es ≤ 7, sí es ≤ 20 → `normal`.

---

3. Errores: try / except

Un error sin manejar **tumba el programa**. Con `try / except` decides qué hacer.

```
def leer_tabla(ruta):
    """Lee un CSV; si no existe, avisa con un mensaje claro."""
    try:
        return pd.read_csv(ruta)
    except FileNotFoundError:
        print("No encontré el archivo:", ruta)
        return None

for ruta in ["data/raw/olist_orders_dataset.csv", "data/raw/no_existe.csv"]:
    tabla = leer_tabla(ruta)
    if tabla is not None:
        print("Leído:", ruta, "→", len(tabla), "filas")
```

- `try:` → "intenta esto".
- `except FileNotFoundError:` → "si el error es *archivo no encontrado*". Se atrapa **un error específico**, no cualquiera: así no escondes otros problemas.
- `return None` → "nada": quien llamó sabe que no hubo tabla.
- `if tabla is not None:` → "si hay tabla".

Otro error típico: texto a número

```
for texto in ["12", "abc", "7"]:
    try:
        print("Número leído:", int(texto))
    except ValueError:
        print("No es un número:", texto)
```

`int("abc")` produce un `ValueError`. El programa **sigue con "7"**: un error en un dato no tumba todo.

¿Por qué importa en un pipeline?

Si procesas mil archivos y el 500 falla sin `try`, pierdes el trabajo de los 499 anteriores. Con `try / except` el programa **avisa bien** y decide qué hacer. Es parte de la rúbrica: la calidad del código incluye el manejo de errores.

Error	Cuándo aparece
`FileNotFoundError`	El archivo no existe
`ValueError`	El valor no sirve (ej. “abc” como número)
`KeyError`	La clave o columna no existe
`ZeroDivisionError`	Dividiste por cero

Errores frecuentes

Qué ves	Por qué pasa	Qué hacer
`IndentationError`	La sangría está mal	Usa 4 espacios y mantén la misma en todo el bloque
`TypeError: ... missing 1 required positional argument`	Llamaste la función con menos datos de los que pide	Revisa los parámetros de `def`
La función imprime `None`	Falta el `return`	Agrega `return resultado`
Nada pasa al “llamar” la función	Solo la definiste	Escribe `nombre_funcion(...)`
`except:` sin tipo	Atrapa todo, incluso errores que querías ver	Indica el tipo (`FileNotFoundError`, `ValueError`...)

Practica tú

1. Escribe una función `doble(x)` que devuelva `x * 2`.
2. Escribe `es_caro(precio)` que devuelva `"caro"` si el precio supera 500 y `"normal"` en otro caso.
3. ¿Qué imprime `clasificar_entrega(20)`? ¿Y `clasificar_entrega(21)`?
4. ¿Qué tipo de error atraparías al leer un CSV que podría no existir?

▼ **Ver respuestas**

1. `def doble(x): return x * 2`
2. `def es_caro(precio):` / `if precio > 500:` / `return "caro"` / `return "normal"`
3. `normal` (20 ≤ 20) y `lenta` (21).
4. `FileNotFoundError`.

Para llevarte

- Una **función** es una receta con nombre: `def`, parámetros, `return`; hay que **llamarla**.
- `if / elif / else` decide; `for` repite; **todo con dos puntos y sangría**.
- `try / except TipoDeError` evita que un error tumbe el programa.